

AIKernel Phase-2 Theory: Semantic Compilation Architecture

A Runtime Theory of Semantic State, Observability, and Governed Transition

Takuya Sogawa

2026-06-06

AIKernel Phase-2 Theory: Semantic Compilation Architecture

A Runtime Theory of Semantic State, Observability, and Governed Transition

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.2.0

Publication date: 2026-06-06

DOI: <https://doi.org/10.5281/zenodo.20312092>

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Canonical language: English

Companion translation: Japanese

Series position: AIKernel / AIOS Phase-2 Theory

Target: AIKernel / AIOS Semantic Runtime

Proposed namespace: AIKernel.Abstractions.Semantics

Stability: Experimental / Non-normative

1. Abstract

This paper defines **Semantic Compilation Architecture** as the Phase-2 theory of AIKernel / AIOS. Phase-1 defines the static architecture of an AI Operating System: knowledge identity, semantic storage, pre-inference admission, and trajectory governance. Phase-2 generalizes these boundaries into a semantic runtime theory that explains how AIOS handles meaning during execution and synthesis.

The central claim of this paper is that AIOS does not execute raw natural-language intent directly. Instead, it transforms human intent into observable semantic states, evaluates semantic transitions under governance constraints, and admits only governed semantic transitions into execution. In this model, LLMs and SLMs are not treated as execution authorities. They are treated as proposal generators for semantic states, transitions, or circuit candidates. AIKernel provides the deterministic governance boundary that evaluates those proposals.

This paper introduces Semantic State, Semantic IR, Governed Circuit, Prototype Space, Semantic Compression, Runtime Policy, and Deterministic Synthesis as first-class architectural concepts. It further connects the Semantic Ellipsoid model from Paper 04 to the quantization stage of Semantic Compilation through a composite distance function whose semantic-state component may be instantiated as a Mahalanobis-style semantic distance.

This paper does not claim that human meaning can be fully formalized, that embedding spaces completely represent human semantics, or that semantic compilation proves factual correctness. It defines a bounded, operational, replayable, and auditable theory for transforming high-entropy human intent into finite governed runtime structures.

2. Position in the AIKernel Phase Structure

AIKernel is organized into three theoretical phases.

Phase-1 = Static Architecture / OS Specification

Phase-2 = Semantic Runtime Theory / Semantic Compilation Architecture

Phase-3 = Foundation / Trust, Proof, Cryptographic Layer

Phase-1 defines the structure of AIOS. Phase-2 defines how AIOS handles meaning. Phase-3 defines how AIOS may become socially and cryptographically trustworthy.

This paper is positioned after the Phase-1 static architecture papers and before the Phase-3 foundation notes. Its purpose is to define the semantic runtime theory that connects AIOS governance boundaries to observable semantic state transitions and future trust-oriented extensions.

2.1 Relationship to Phase-1

Phase-1 defines four static architectural boundaries.

Phase-1 paper	Boundary	Role from Phase-2
Paper 01: ROM Format and Knowledge Snapshot Model	Knowledge identity	Immutable knowledge units referenced by semantic state
Paper 02: VFS Architecture and Semantic Storage Model	Storage boundary	Semantic storage for persisting and retrieving state artifacts
Paper 03: Pre-Inference Admissibility Governance	Entry boundary	Deterministic admission before semantic transition begins
Paper 04: Trajectory Governance Model	Runtime governance	Operational evaluation of semantic trajectories and candidate actions

Phase-2 generalizes these boundaries into a semantic runtime. In particular, the four slots of Semantic IR are deliberately aligned with the four Phase-1 boundaries.

Semantic IR = (G, T, C, B)

G = Graph topology -> Paper 01 / ROM dependency topology

T = Capability types -> Paper 02 / VFS and provider capabilities

C = Governance constraints -> Paper 03 / admissibility and policy constraints

B = Boundary invariants -> Paper 04 / trajectory and runtime safety invariants

This alignment is not merely documentary. It means that Phase-1 provides the static container required for Phase-2 to compile meaning into governed runtime structures.

2.2 Relationship to Paper 04

Paper 04 defines an operational governance model for evaluating inference trajectories using scores, thresholds, risk signals, and ReplayLog records. This paper generalizes that operational model into a semantic runtime theory, where semantic states, observability, and governed transitions are treated as first-class architectural concepts.

Paper 04:

Trajectory Governance operational model

- Scores
- Thresholds
- Risk evaluation
- Abort / Permit / Clarify
- ReplayLog

Phase-2:

Semantic Runtime Theory

- Semantic State
- Semantic Transition

- Observability Primitive
- Runtime Policy
- Semantic Compilation

Paper 04 is a governance algorithm. Phase-2 is the theoretical model of the semantic runtime in which such algorithms operate.

2.3 Relationship to Phase-3

Phase-3 is not required for the validity of the Phase-2 theory. It is introduced as a future foundation layer for identity, provenance, cryptographic addressability, digital deeds, and trust-oriented AIOS extensions.

The concepts defined in this paper - Semantic State, Semantic IR, Governed Circuit, Runtime Policy, and ReplayLog - may later be extended into Phase-3 topics such as ROM Identity, Digital Deed, PQC Address, Anti-Slop Architecture, and provenance verification. This paper, however, does not attempt to solve those Phase-3 problems.

3. Problem Statement: From Prompt Execution to Semantic Runtime

Many LLM applications and agent frameworks connect natural-language prompts directly to execution planning, tool selection, code generation, or external system operations. In such systems, a high-entropy natural-language request is passed to a stochastic generator, and the generated output may then drive system boundaries.

This creates several structural problems.

1. **Absence of semantic boundaries:** The system does not reliably distinguish explanation, planning, execution, destructive intent, or hypothetical reasoning.
2. **Unobservable state transition:** The system cannot explicitly observe which semantic state it is leaving or entering.
3. **Confusion between prompt and specification:** A prompt is a high-entropy expression of intent, yet it is often treated as an executable specification.
4. **Non-deterministic synthesis:** Tool graphs, workflows, code structures, or agent loops may be constructed probabilistically.
5. **Lack of replayable semantic transformation:** The system cannot later reconstruct why a human intent was mapped to a particular execution topology.

AIKernel Phase-2 treats this as a transition from prompt execution to semantic runtime. AIOS does not execute raw natural-language intent directly. It compiles human intent into observable, governable, and replayable semantic transitions.

This can be summarized as follows.

Observability -> Governability -> Replayability

A semantic state that cannot be observed cannot be governed. A transition that cannot be governed cannot be replayed as auditable system evidence. Phase-2 therefore begins by defining what can be observed.

4. Scope and Non-Claims

4.1 Scope

This paper defines the following concepts.

- Semantic State
- Semantic IR
- Semantic Transition
- Observability Primitive
- Governed Circuit and Prototype Space
- Semantic Compression / Semantic Quantization
- Runtime Policy and Admissibility

- Deterministic Synthesis under fixed mappings
- Relationship to Phase-1 static architecture
- Relationship to future Phase-3 foundation notes

4.2 Non-Claims

This paper does not claim the following.

- Human meaning can be completely formalized.
- Semantic Ellipsoids fully represent human semantics.
- Semantic IR losslessly captures all natural-language intent.
- Semantic transitions are globally mathematically stable.
- LLM internal inference can be made fully deterministic.
- Phase-3 cryptographic trust, PQC addressing, Digital Deed, or provenance is solved here.
- Quantum computing is required for the correctness of Semantic Compilation.

The claim is narrower: AIKernel can transform meaning at its governance boundary into observable, constrained, replayable, and bounded semantic runtime artifacts.

5. Semantic Entropy as an Operational Proxy

Human intent expressed in natural language may imply many interpretations, tool choices, data dependencies, and execution paths. For an input intent u , let the set of reachable admissible semantic execution trajectories be represented as:

$\text{Traj_adm}(u) = \{ \text{admissible semantic execution trajectories reachable from } u \}$

The semantic entropy $H_{sem}(u)$ is not used here as a strict Shannon entropy over human meaning. Instead, it is an operational proxy for the diversity of admissible trajectories available at the AIKernel governance boundary.

$H_{sem}(u) \sim \log |\text{Traj_adm}(u)|$

The purpose of Semantic Compilation is not to prove that human meaning has been reduced to a single philosophical interpretation. Its purpose is to reduce the operational uncertainty of execution topology under explicit governance constraints.

5.1 High-Entropy Intent

An unstructured prompt may expand into many interpretations.

User intent

- > multiple interpretations
- > multiple tools
- > multiple context scopes
- > multiple execution paths

If such a prompt is delegated directly to an unconstrained LLM agent loop, the system may exhibit trajectory drift, privilege escalation, unbounded tool invocation, or unstable workflow construction.

5.2 Bounded Semantic Runtime

AIKernel does not claim that semantic entropy is literally reduced to zero in a philosophical sense. It claims that the execution-relevant uncertainty is bounded at the runtime boundary. Natural-language intent is progressively mapped into Semantic IR, Prototype Space, Governed Circuits, and Runtime Policy, producing a finite set of auditable transition candidates.

6. Semantic State Model

A semantic runtime requires an observable representation of meaning. AIKernel models an observable semantic state as an implementation-oriented approximation.

$$s_t = (\mu_t, \Sigma_t)$$

Here:

- μ_t is the center vector or representative semantic embedding for state t .
- Σ_t is an uncertainty estimate, usually approximated as a diagonal covariance matrix in implementation.

This model inherits the caution defined in Paper 04: an embedding space is not assumed to perfectly represent human meaning. A semantic state is an observability primitive, not a metaphysical definition of meaning. It is an approximation introduced so that semantic variation can be observed, constrained, compared, and replayed at the AIOS governance boundary.

6.1 Semantic Ellipsoid

The Semantic Ellipsoid is the admissible region around a semantic state. It represents observed semantic variance rather than perfect meaning.

$$E_t(k) = \{x \in V \mid d(x, s_t) \leq k\}$$

In implementation, the distance component may be approximated using diagonal variance and a stabilizing constant.

$$d(x, s_t) = \sqrt{\sum_j ((x_j - \mu_{t,j})^2 / (\sigma_{t,j}^2 + \epsilon))}$$

This formulation gives AIKernel an operational method for detecting whether a transition remains within a governed semantic region.

6.2 From Observability to Compilation

Paper 04 uses Semantic Ellipsoids to observe and govern inference trajectories. Phase-2 reuses that observability principle at a higher level: the same idea becomes part of the compatibility evaluation between an unverified Semantic IR and a candidate Governed Circuit.

Observability becomes compilability.

This is the central bridge between Phase-1 runtime governance and Phase-2 semantic compilation.

7. Semantic IR

A high-entropy concept specification must be mapped into a deterministic intermediate representation before it can be governed. This paper defines **Semantic IR** as a four-slot tuple.

$$X = G \times T \times C \times B$$

$$x = (G, T, C, B)$$

The four slots are:

Slot	Name	Meaning	Phase-1 correspondence
G	Graph Topology	Dataflow, reasoning dependencies, and execution topology	Paper 01
T	Capability Types	Required providers, tools, VFS access, and solver capabilities	Paper 02

Slot	Name	Meaning	Phase-1 correspondence
C	Governance Constraints	Policy limits, risk thresholds, budgets, admissibility rules	Paper 03
B	Boundary Invariants	Fail-closed conditions, preconditions, postconditions, runtime invariants	Paper 04

The Semantic IR is not a natural-language summary. It is a governed structural representation that can be compared, constrained, synthesized, and replayed.

7.1 Why Four Slots

The four-slot structure is designed to separate concerns that should not be collapsed into a single prompt.

- G describes what the execution topology is.
- T describes what capabilities are required.
- C describes which policies constrain execution.
- B describes which invariants must never be violated.

This separation makes admission, governance, synthesis, and replay explicit.

7.2 Associated Semantic State

Semantic IR remains a discrete structural artifact. The observable semantic state is associated with the IR but is not added as a fifth IR slot.

Semantic IR: $x = (G, T, C, B)$

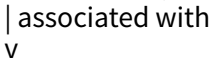
Associated semantic state: $s = (\mu, \Sigma)$

This distinction is important. The structural IR contains topology, capabilities, constraints, and invariants. The semantic state represents observable uncertainty around the meaning associated with that IR.

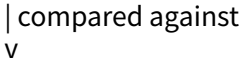
Concept specification / human intent



Semantic IR: $x = (G, T, C, B)$ [discrete structural artifact]



Semantic state: $s = (\mu, \Sigma)$ [observable semantic uncertainty]



Governed circuit: $c_k = (G_k, T_k, C_k, B_k)$ with s_k

This diagram is intentionally not a dataflow graph. It clarifies that Semantic IR and Semantic State are associated but distinct artifacts: one is the structural compilation target, and the other is the observability model used to evaluate compatibility and uncertainty.

8. Governed Circuits and Prototype Space

A **Governed Circuit** is a formally admissible, bounded, fail-closed semantic execution topology.

Unlike an unconstrained agent loop, a Governed Circuit is a pre-verified execution structure with explicit capabilities, policies, and boundary invariants. Examples include retrieval circuits, quarantine circuits, cross-verification circuits, deterministic solver circuits, and read-only summarization circuits.

The Prototype Space P is a finite set of such verified circuits.

$P = \{c_1, c_2, \dots, c_N\} \text{ subset } X$

Each candidate circuit is represented in the same four-slot Semantic IR space.

$c_k = (G_k, T_k, C_k, B_k) \text{ in } X$

This property allows an unverified input IR x and a verified circuit candidate c_k to be compared inside the same structural space.

8.1 Closed but Extensible

For safety, the Prototype Space is closed during a single compilation decision. A compiler pass must select from currently verified circuits only. However, over longer governance cycles, new circuits may be proposed, verified, logged, and added to the prototype set through future Foundation mechanisms.

This trade-off is intentional.

Expression freedom $\leftrightarrow$ systemic safety

Phase-2 prioritizes bounded deterministic governance over unconstrained expressive generation.

9. Semantic Transition Model

A Semantic Transition is a candidate movement from one semantic state or structural IR to another.

$x_t \rightarrow x_{t+1}$

$s_t \rightarrow s_{t+1}$

In AIKernel, the LLM / SLM may propose such transitions, but it cannot commit them. The Kernel evaluates the transition under runtime policy.

Propose $\rightarrow$ Evaluate $\rightarrow$ Decide $\rightarrow$ Commit

- **Propose:** LLM / SLM proposes a semantic state, IR, circuit, or action.
- **Evaluate:** AIKernel evaluates admissibility, distance, capability compatibility, policy constraints, and invariants.
- **Decide:** PDP returns a controlled decision.
- **Commit:** PEP commits only permitted transitions.

9.1 Governed Transition

A governed transition is a semantic transition that has passed policy evaluation and is permitted to affect executable system state.

GovernedTransition(x, c_k) = true

only if admissibility, compatibility, and invariants hold

This model preserves the Phase-1 principle that stochastic inference may propose candidates but never directly owns execution authority.

10. Semantic Compression and Admissibility

Semantic Compilation maps a high-entropy Semantic IR into a finite governed circuit candidate. This mapping is called **Semantic Compression** or **Semantic Quantization**.

10.1 Admissibility Function

The compiler evaluates whether an input IR x may be projected onto a candidate circuit c_k .

$A(x, c_k) = 1$
iff constraints are preserved
and invariants are preserved
and capabilities are compatible

More explicitly:

$A(x, c_k) = 1$ if
 $x.C \subseteq c_k.C$
and $x.B \Rightarrow c_k.B$
and $c_k.T$ proves $x.T$
otherwise $A(x, c_k) = 0$

If no candidate circuit satisfies the admissibility function, the compiler raises an Admissibility Error and fails closed.

forall c_k in P , $A(x, c_k) = 0$
 $\Rightarrow$ AdmissibilityError
 $\Rightarrow$ FailClosed

10.2 Composite Distance Function

Given the admissible candidate set, the compiler selects the closest governed circuit under a composite distance function.

$Q(x) = \operatorname{argmin}_{\{c_k \in P, A(x, c_k)=1\}} d(x, c_k)$

The distance function $d(x, c_k)$ should not be understood as a single Euclidean distance. Semantic IR consists of heterogeneous components: graph topology, capability requirements, governance constraints, and boundary invariants. In addition, each IR is associated with an observable semantic state.

Therefore, this paper defines $d(x, c_k)$ as a composite distance function. More precisely, it is a heterogeneous metric aggregation: each component is evaluated in its own appropriate structural or semantic space, and the weighted sum is used as an operational ranking score rather than a single homogeneous metric.

$d(x, c_k)$
 $= \lambda_G d_G(G, G_k)$
 $+ \lambda_T d_T(T, T_k)$
 $+ \lambda_C d_C(C, C_k)$
 $+ \lambda_B d_B(B, B_k)$
 $+ \lambda_S d_S(s, s_k)$

Here:

- d_G measures graph topology difference.
- d_T measures capability requirement difference.
- d_C measures governance constraint difference.
- d_B measures boundary invariant difference.
- d_S measures observable semantic-state difference.

The semantic-state component may be instantiated as a Mahalanobis-style semantic distance, but the composite distance function itself is not a single metric over a homogeneous space.

$d_S(s, s_k)$
 $= \sqrt{(\mu - \mu_k)^T (\Sigma_k + \epsilon I)^{-1} (\mu - \mu_k)}$

Here, $\epsilon > 0$ is a stabilizing constant and I is the identity matrix.

This formulation separates discrete structural differences from continuous semantic uncertainty. It prevents graph topology, capability authorization, policy constraints, invariants, and semantic uncertainty from being collapsed into a single undifferentiated vector distance.

The d_S component is an observability-oriented approximation, not a proof of semantic completeness. It evaluates compatibility between an observable semantic state and a governed circuit candidate; it does not prove that human meaning has been completely measured.

10.3 Terminology: Quantization Is Not Quantum Mechanics

The term quantization is used here in the information-theoretic and signal-processing sense: mapping a continuous or high-cardinality space into a finite set of discrete representatives. It does not refer to physical quantum mechanics.

11. Deterministic Synthesis

After a candidate circuit c_k is selected, the synthesis layer materializes the structural IR and selected circuit into runtime artifacts such as interface contracts, dependency injection registrations, static SGP graphs, tool access policies, and replay configuration.

Let the synthesis function be:

$$S : X \times P \rightarrow R$$

where R is the runtime artifact space.

11.1 Theorem 1: Deterministic Structural Synthesis

Given fixed input IR, fixed selected circuit, fixed compiler version, fixed policy set, fixed mapping rules, fixed templates, and fixed runtime environment configuration, the synthesis output is reproducible.

$$\text{Fixed}(x, c_k, \text{compiler}, \text{policies}, \text{mappings}, \text{templates}) \\ \Rightarrow \text{Var}(S(x, c_k)) = 0$$

Proof sketch: Once the stochastic extraction step has produced a recorded Semantic IR, and once quantization selects a recorded candidate circuit, synthesis is implemented as a deterministic mapping from structured data to runtime artifacts. It does not call a stochastic text generator during the synthesis step. Therefore, under fixed implementation and environment conditions, the output structure is reproducible.

This theorem does not claim that the original natural-language extraction is perfect. It claims that the synthesis step is structurally deterministic after the IR and circuit have been fixed.

11.2 Compiler Correctness as Constraint Preservation

Classical compiler correctness often concerns semantic preservation from source program to target program. In AIKernel, the claim must be more cautious. Human intent is not fully formalized as source code. Therefore, Phase-2 defines compiler correctness as preservation of enforceable constraints, capabilities, and boundary invariants.

$$\text{Preserve}(C, B, T) \text{ across } S(x, c_k)$$

The compiler is correct for a given governance profile if the synthesized runtime artifact enforces the constraints C , capabilities T , and invariants B selected during compilation.

This is not a proof that all human meaning has been preserved. It is a proof obligation that execution-relevant governance constraints are preserved.

12. Runtime Policy

Runtime Policy is the set of rules that determines whether semantic transitions, IR projections, and circuit selections are admissible.

Runtime Policy includes:

- capability requirements;
- risk thresholds;
- token and latency budgets;
- isolation requirements;
- context provenance requirements;

- fail-closed triggers;
- replay requirements;
- allowed circuit families;
- forbidden transition patterns.

A policy may reject a transition, require clarification, downgrade capabilities, require a snapshot, route to a deterministic solver, or admit the transition into a governed circuit.

Runtime Policy is not free-form natural language. It must be represented as versioned, replayable, machine-checkable policy artifacts.

13. Relationship to Existing Compiler Architectures

Semantic Compilation is inspired by compiler architecture, but it is not strictly isomorphic to LLVM, Roslyn, or any specific compiler implementation.

The analogy is architectural.

Classical compiler	Semantic compiler
Source code	Concept specification / human intent
Syntax parsing	Semantic extraction
AST / IR	Semantic IR
Optimization	Semantic quantization and policy refinement
Backend	Governed circuit synthesis
Machine code	AIKernel runtime artifact

The purpose of this analogy is to clarify the role of intermediate representation, admissibility errors, deterministic synthesis, and backend selection. It does not claim that natural-language semantics has the same formal properties as a programming language grammar.

14. Optional Backend Optimization

As the Prototype Space grows, searching for the best governed circuit may become computationally expensive. Graph topology matching, subgraph compatibility, and multi-dimensional constraint optimization may become difficult for large enterprise systems.

This paper allows optional backend optimization mechanisms for the quantization step. Such mechanisms may include classical solvers, heuristic graph search, SMT solvers, integer programming, or future quantum-inspired optimization methods.

These backends are optimization mechanisms only. They do not replace the Semantic IR, Runtime Policy, Admissibility Function, or deterministic synthesis layer. Even if a probabilistic or approximate optimizer is used to propose candidate circuits, the final admission and synthesis decisions remain governed by deterministic validation rules.

15. Applications

15.1 Governed Enterprise Workflow Synthesis

When a user requests a complex enterprise workflow, such as a multi-department audit over financial spreadsheets and compliance documents, the Semantic Compiler does not instantiate an unconstrained autonomous agent loop. Instead, it extracts a Semantic IR and maps it to a composition of governed circuits.

Retrieval Circuit
 -> Quarantine Circuit

- > Cross-Verification Circuit
- > Report Synthesis Circuit

Capabilities, storage boundaries, risk thresholds, and replay requirements are locked before execution begins.

15.2 Re-Quantization Under Policy Change

When corporate compliance rules, ROM assets, or VFS capabilities change, the admissibility of existing Semantic IR artifacts may change. AIKernel can re-evaluate $A(x, c_k)$ under the new policy set and re-quantize the IR to another governed circuit if required.

This provides a structured model for policy-aware runtime evolution without relying on ad-hoc prompt rewriting.

16. Theoretical Guarantees and Non-Guarantees

16.1 Guarantees Under Fixed Conditions

Under fixed versions, policies, mappings, templates, and recorded inputs, AIKernel can guarantee:

- deterministic structural synthesis;
- fail-closed rejection when no admissible circuit exists;
- preservation of declared capabilities, constraints, and invariants in generated runtime artifacts;
- replayability of circuit selection and synthesis decisions;
- auditable traceability from intent extraction to runtime artifact.

16.2 Non-Guarantees

AIKernel does not guarantee:

- perfect extraction of human intent;
- factual correctness of generated content;
- complete semantic equivalence between natural language and runtime artifacts;
- global mathematical stability of semantic transitions;
- complete immunity to adversarial inputs;
- correctness of future Phase-3 cryptographic or provenance mechanisms.

This bounded-claim structure is intentional. Phase-2 defines a governance-oriented semantic runtime, not a universal theory of meaning.

17. Limitations and Future Work

17.1 Prototype Expansion Problem

A finite Prototype Space increases safety but reduces expressive freedom. Novel user intents may be compressed into conservative circuits, causing loss of expressiveness. Future work should define safe mechanisms for proposing, verifying, logging, and admitting new circuit prototypes.

17.2 Extraction Uncertainty

The extraction from natural-language intent to Semantic IR may itself be uncertain. When extraction cannot be performed deterministically or with sufficient confidence, the compiler must return Clarify or fail closed.

17.3 Proof-Carrying Synthesis

Future work may attach machine-checkable proofs or signed attestations to synthesized runtime artifacts. This would connect Phase-2 to Phase-3 Foundation topics such as Digital Deed, provenance, ROM Identity, and cryptographic addressability.

17.4 Runtime Calibration

ReplayLog data may be used to calibrate distance weights, admissibility thresholds, and circuit selection rules. Calibration must not override hard policy constraints or fail-closed boundaries.

18. Conclusion

Semantic Compilation Architecture defines the Phase-2 theory of AIKernel / AIOS. It explains how the static boundaries defined in Phase-1 become a runtime theory of meaning, observability, and governed transition.

The core contribution of this paper is the shift from prompt execution to semantic compilation. AIKernel does not treat natural language as directly executable. It transforms high-entropy human intent into Semantic IR, evaluates its compatibility with a finite Prototype Space of Governed Circuits, and synthesizes deterministic runtime artifacts under explicit Runtime Policy.

This model does not make stochastic intelligence deterministic. It places stochastic proposals behind deterministic governance boundaries. It does not prove the completeness of meaning. It defines the operational structures required to observe, govern, replay, and audit meaning-bearing transitions.

In this sense, Phase-2 connects Phase-1 to Phase-3. It takes the static OS boundaries of AIKernel and turns them into a semantic runtime theory capable of future extension into provenance, identity, proof, and cryptographic trust.

References

1. Shannon, Claude E. "A Mathematical Theory of Communication." Bell System Technical Journal, vol. 27, no. 3, 1948, pp. 379-423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.
2. Mahalanobis, Prasanta Chandra. "On the Generalized Distance in Statistics." Proceedings of the National Institute of Sciences of India, vol. 2, no. 1, 1936, pp. 49-55. Available at: <https://insa.nic.in/writereaddata/UploadedFiles/>
3. Lattner, Chris, and Vikram Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." International Symposium on Code Generation and Optimization (CGO 2004), 2004, pp. 75-88. DOI: 10.1109/CGO.2004.1281665.
4. Sikka, Varin, and Vishal Sikka. "Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models." arXiv:2507.07505, 2025. DOI: 10.48550/arXiv.2507.07505.
5. National Institute of Standards and Technology. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162.
6. OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013. Available at: <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>.
7. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model." Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
8. Sogawa, Takuya. "AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model." Zenodo, 2026. DOI: 10.5281/zenodo.20306539.
9. Sogawa, Takuya. "AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model." Zenodo, 2026. DOI: 10.5281/zenodo.20307891.
10. Sogawa, Takuya. "AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance." Zenodo, 2026. DOI: 10.5281/zenodo.20308639.
11. Sogawa, Takuya. "AIKernel Phase-1 Paper 04: Trajectory Governance Model." Zenodo, 2026. DOI: 10.5281/zenodo.20309510.